

CSC 3201 Module 4 Assignment - Hashing and Passwords

Marc Helwee and Jaxon Bruzgul

Task 1: Exploring Pseudo-Randomness and Collision Resistance

What We Implemented

We implemented SHA-256 hashing for arbitrary inputs and displayed the complete 256-bit digests in hexadecimal format. We then created pairs of ASCII strings that differ in exactly one bit. The implementation changes only bit 0x20 in one letter, which changes its case while preserving every other bit. For each pair, the program verifies that the input Hamming distance is exactly one bit, hashes both strings with SHA-256, and counts the number of differing bytes in the two 32-byte digests.

We also implemented a birthday-collision search for truncated SHA-256 digests. The program truncates the leftmost 8 through 50 bits of SHA-256 and stores previously seen truncated digests in a dictionary. When a repeated truncated digest is found for two distinct inputs, the program records both inputs, the number of inputs tried, and the elapsed time.

The collision program creates the following files:

collision_results.csv, containing the collision data for every digest size from 8 through 50 bits.

collision_time.png, plotting digest size against collision time.

collision_inputs.png, plotting digest size against the number of inputs tried, including the theoretical birthday estimate.

Test Observations

The SHA-256 portion of the program produced the following complete digests:

Input: Hello, World!

SHA256: dffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f

Input: Python

SHA256: 18885f27b5af9012df19e496460f9294d5ab76128824c6f993787004f6d9a7db

Input: Cryptography

SHA256:

b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857

For the one-bit-input tests, the program prints three randomly generated pairs. Each pair has an input Hamming distance of exactly one bit. The resulting SHA-256 digests are completely different-looking hexadecimal values, and nearly all of the 32 digest bytes differ. This demonstrates the avalanche effect: a one-bit change in the input causes a large, unpredictable-looking change in the hash output.

Collision Measurements

Paste the full table generated in `collision_results.csv` here after running `task1_hashing.py`.

Also insert these two generated graphs here:

`collision_time.png` — Digest Size vs. Collision Time

`collision_inputs.png` — Digest Size vs. Number of Inputs

The collision search becomes much more expensive as the digest length increases. Small truncated digests collide quickly, while a 50-bit collision normally requires tens of millions of inputs. The measured number of inputs varies because the inputs are random, but the general growth follows the birthday bound.

Question 1: What do you observe based on Task 1b? How many bytes are different between the two digests?

The one-bit input change produces a drastically different SHA-256 digest. This is the avalanche effect. SHA-256 is designed so that an attacker cannot predict how an input change will affect the output. Although the inputs differ by only one bit, nearly all of the 32 bytes in the two digests differ. For independent-looking hash outputs, each output byte has only a $1/256$ chance of matching the corresponding byte in the other digest, so approximately 31.875 out of 32 bytes are expected to differ.

Question 2: What is the maximum number of files you would ever need to hash to find a collision on an n -bit digest? Given the birthday bound, what is the expected number of hashes before a collision on an n -bit digest? Is this what you observed? Based on the

data you have collected, speculate on how long it might take to find a collision on the full 256-bit digest.

There are only 2^n possible n -bit digest values. By the pigeonhole principle, hashing $2^n + 1$ distinct files guarantees at least one collision.

The birthday bound is much smaller than this worst-case number. The expected number of hashes before a collision is approximately:

$$1.25 \times 2^{(n/2)}$$

The measured results fluctuate above and below the birthday estimate because each run is random, but the number of inputs grows approximately with $2^{(n/2)}$. This is why a 50-bit collision is difficult but still potentially feasible on a normal computer, while a full SHA-256 collision is not.

For a 256-bit SHA-256 digest, the birthday estimate is approximately 1.25×2^{128} hashes. Even at an unrealistically fast rate of one billion hashes per second, this would require roughly 1.35×10^{22} years. Therefore, a brute-force collision attack against full SHA-256 is infeasible.

Question 3: Given an 8-bit digest, would you be able to break the one-way property by finding a pre-image? Do you think this would be easier or harder than finding a collision? Why or why not?

Yes. For an 8-bit digest, there are only 256 possible outputs. To find a pre-image for a specified 8-bit target digest, an attacker can hash candidate messages until one produces the target. Each guess has probability $1/256$ of succeeding, so the expected work is about 256 hashes.

Finding a pre-image is harder than finding a collision. A collision can be found with the birthday attack in approximately $1.25 \times 2^{(8/2)}$, or about 20 hashes on average. A pre-image attack must match one specific target digest, while a collision attack can use any two different messages that produce the same digest.

Task 2: Breaking Real Hashes

What We Implemented

We implemented a custom bcrypt dictionary-cracking program for the supplied shadow file. The program reads entries in the form:

User:\$Algorithm\$Workfactor\$SaltHash

It separates each bcrypt value into its algorithm, work factor, 22-character salt, and remaining hash value. The program uses only words from the NLTK word corpus that are between 6 and 10 letters long, which matches the assignment constraints.

The cracking code uses multiprocessing to check portions of the dictionary in parallel. It also groups hashes with the same bcrypt work factor and salt. Since hashes with the same salt can reuse the same bcrypt calculation for each candidate word, this avoids repeating unnecessary work.

Test Observations

The recorded cracking results were:

User	Algo	Cost	Salt	Password	Time (s)	Guesses
Bilbo	2b	08	J9FW66ZdPI2nrIMcOxFYI.	welcome	9499.14	131200
Gandalf	2b	08	J9FW66ZdPI2nrIMcOxFYI.	wizard	9501.60	132400
Thorin	2b	08	J9FW66ZdPI2nrIMcOxFYI.	diamond	2647.42	31400
Fili	2b	09	M9xNRFBdN0pUkPKIVCSBzu	desire	2612.15	31000
Kili	2b	09	M9xNRFBdN0pUkPKIVCSBzu	ossify	8091.44	76600
Balin	2b	10	xGKjb94iwmlth954hEaw3O	hangout	5650.79	49600
Dwalin	2b	10	xGKjb94iwmlth954hEaw3O	drossy	3552.83	34400
Oin	2b	10	xGKjb94iwmlth954hEaw3O	ispaghul	6831.54	57400
Gloin	2b	11	/8UByex2ktrWATZOBLZ0Du	oversave	8215.17	78400
Dori	2b	11	/8UByex2ktrWATZOBLZ0Du	indoxylic	6683.07	55200
Nori	2b	11	/8UByex2ktrWATZOBLZ0Du	swagsman	9390.48	111400
Ori	2b	12	rMeWZtAVcGHLEiDNeKCz8O	airway	473.61	2800
Bifur	2b	12	rMeWZtAVcGHLEiDNeKCz8O	corrosible	2351.56	26200
Bofur	2b	12	rMeWZtAVcGHLEiDNeKCz8O	libellate	7277.19	62000
Durin	2b	13	6ypcazOOKUT/a7EwMuljH.	purrone	8916.14	90400

Total wall-clock time: 9501.88s

All work factors from 8 through 13 were solved. The exact cracking times are not ordered solely by work factor because a password found early in the word list finishes sooner than a password found near the end. However, each increase in bcrypt work

factor makes every individual bcrypt attempt more expensive. This intentional slowness is why bcrypt is more appropriate for passwords than a fast hash such as SHA-256.

Question 4: Given your results, how long would it take to brute force a password in the format word1, word1:word2, and word1:word2?

The assignment states that the filtered NLTK corpus contains approximately 135,000 possible words. The recorded work-factor-8 results processed about 132,400 guesses in 9,501.60 seconds, which is approximately 13.9 candidate passwords per second. The following estimates use that observed rate. They are optimistic because higher bcrypt work factors take longer.

For word1:word2, the number of possible passwords is:

$$135,000 \times 135,000 = 18,225,000,000 \text{ combinations}$$

At approximately 13.9 guesses per second, the worst-case time is about 41 years. On average, the password would be found halfway through the search, or about 20.5 years.

For word1:word2:word3, the number of possible passwords is:

$$135,000^3 = 2,460,375,000,000,000 \text{ combinations}$$

At the same rate, the worst-case time is about 5.6 million years. The average search time is about 2.8 million years.

For word1:word2:number, the number can contain from one through five digits. There are:

$$9 + 90 + 900 + 9,000 + 90,000 = 99,999 \text{ possible numbers}$$

Therefore, the number of possible passwords is:

$$135,000 \times 135,000 \times 99,999 \\ = 1,822,481,775,000,000 \text{ combinations}$$

At approximately 13.9 guesses per second, the worst-case time is about 4.1 million years, with an average time of about 2.1 million years.

These estimates show why combining multiple words and a number creates much stronger passwords than a single dictionary word. The number of candidates grows multiplicatively, while bcrypt ensures that each candidate attempt remains slow.

Interesting Observations

The first task demonstrates that SHA-256 behaves like a pseudorandom function: changing one input bit creates a digest with no visible relationship to the original digest. The second task demonstrates two different security properties. Fast hashes such as SHA-256 are useful for integrity checking but are not suitable for password storage because attackers can try guesses quickly. Bcrypt intentionally increases the cost of every password guess, and its work factor can be raised over time as computer hardware improves.

The salt in bcrypt does not need to be secret. Its purpose is to ensure that identical passwords normally produce different stored hashes and to prevent attackers from reusing one precomputed table against every user. The supplied exercise includes groups that share salts so that the effect of grouping repeated bcrypt work can be demonstrated by the cracking program.

CODE

Task1_hashing.py

```
import csv
import hashlib
import random
import string
import time

def _to_bytes(input_data):
    """Convert a string or byte sequence into bytes for SHA-256."""
    if isinstance(input_data, str):
        return input_data.encode("utf-8")
    return bytes(input_data)

def sha256_hash(input_data):
    """Return the complete SHA-256 digest in hexadecimal."""
    return hashlib.sha256(_to_bytes(input_data)).hexdigest()

def truncated_hash(input_data, bits):
    """Return the leftmost requested number of bits from a SHA-256 digest."""
    if not 8 <= bits <= 256:
        raise ValueError("bits must be between 8 and 256")

    digest = hashlib.sha256(_to_bytes(input_data)).digest()
    byte_count = (bits + 7) // 8
```

```

value = int.from_bytes(digest[:byte_count], "big")
return value >> (8 * byte_count - bits)

```

```

def bit_hamming_distance(value1, value2):
    """Return the number of differing bits between equal-length byte strings."""
    data1 = _to_bytes(value1)
    data2 = _to_bytes(value2)

    if len(data1) != len(data2):
        raise ValueError("inputs must have equal length")

    return sum(
        (byte1 ^ byte2).bit_count()
        for byte1, byte2 in zip(data1, data2)
    )

```

```

def differing_digest_bytes(hash1, hash2):
    """Return how many of the 32 SHA-256 digest bytes differ."""
    digest1 = bytes.fromhex(hash1)
    digest2 = bytes.fromhex(hash2)
    return sum(
        byte1 != byte2
        for byte1, byte2 in zip(digest1, digest2)
    )

```

```

def random_string(length=10):
    return "".join(random.choice(string.ascii_letters) for _ in range(length))

```

```

def find_hamming_distance_1():
    """
    Return two readable ASCII strings whose encoded bytes differ in exactly
    one bit. Toggling 0x20 changes only the case of one letter.
    """
    base = random_string(10)
    index = random.randrange(len(base))

    changed = bytearray(base.encode("ascii"))
    changed[index] ^= 0x20
    modified = changed.decode("ascii")

    assert bit_hamming_distance(base, modified) == 1
    return base, modified

```

```

def find_collision(bits):
    """
    Find a birthday collision for a truncated SHA-256 digest.

    Inputs are distinct 8-byte counter values. There is no short attempt cap
    because a 50-bit birthday collision normally needs tens of millions of
    inputs.
    """
    seen = {}
    start = time.perf_counter()
    attempts = 0

    while True:

```

```

message = attempts.to_bytes(8, "big")
digest = truncated_hash(message, bits)
previous = seen.get(digest)
attempts += 1

```

```

if previous is not None:
    elapsed = time.perf_counter() - start
    return (
        previous.to_bytes(8, "big"),
        message,
        digest,
        attempts,
        elapsed,
    )

```

```

seen[digest] = attempts - 1

```

```

def task_1a():
    print("Task 1a: SHA-256 hashes of arbitrary inputs")

    for text in ["Hello, World!", "Python", "Cryptography"]:
        print(f"Input: {text}")
        print(f"SHA256: {sha256_hash(text)}")

    print()

```

```

def task_1b():
    print("Task 1b: Inputs with Hamming distance exactly 1 bit")

    for pair_number in range(1, 4):
        input1, input2 = find_hamming_distance_1()
        hash1 = sha256_hash(input1)
        hash2 = sha256_hash(input2)

        print(f"Pair {pair_number}:")
        print(f" Input 1: {input1!r}")
        print(f" Input 2: {input2!r}")
        print(
            f" Input Hamming distance: "
            f"{bit_hamming_distance(input1, input2)} bit"
        )
        print(f" SHA256(Input 1): {hash1}")
        print(f" SHA256(Input 2): {hash2}")
        print(
            f" Different digest bytes: "
            f"{differing_digest_bytes(hash1, hash2)} of 32"
        )

    print()

```

```

def save_results(rows):
    with open("collision_results.csv", "w", newline="") as output_file:
        writer = csv.writer(output_file)
        writer.writerow([
            "digest_bits",
            "input_1_hex",
            "input_2_hex",
            "truncated_digest_bits",

```

```

    "inputs_tried",
    "collision_time_seconds",
])

```

```

for bits, input1, input2, digest, attempts, elapsed in rows:

```

```

    writer.writerow([
        bits,
        input1.hex(),
        input2.hex(),
        format(digest, f'0{bits}b'),
        attempts,
        f'{elapsed:.6f}',
    ])

```

```

print("\nSaved complete measurements as collision_results.csv")

```

```

def plot_results(bits_list, time_list, inputs_list):

```

```

    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

```

```

    figure1, axis1 = plt.subplots(figsize=(7, 5))
    axis1.plot(bits_list, time_list, "o-", color="tab:blue")
    axis1.set_xlabel("Digest size (bits)")
    axis1.set_ylabel("Collision time (seconds)")
    axis1.set_title("Digest Size vs. Collision Time")
    axis1.set_yscale("log")
    axis1.grid(True, which="both", alpha=0.3)
    figure1.tight_layout()
    figure1.savefig("collision_time.png", dpi=150)

```

```

    figure2, axis2 = plt.subplots(figsize=(7, 5))
    axis2.plot(
        bits_list,
        inputs_list,
        "s-",
        color="tab:red",
        label="Measured inputs",
    )

```

```

    birthday_estimate = [2 ** (bits / 2) for bits in bits_list]

```

```

    axis2.plot(
        bits_list,
        birthday_estimate,
        "--",
        color="gray",
        label="2^(n/2) birthday estimate",
    )

```

```

    axis2.set_xlabel("Digest size (bits)")
    axis2.set_ylabel("Number of inputs tried")
    axis2.set_title("Digest Size vs. Number of Inputs")
    axis2.set_yscale("log")
    axis2.legend()
    axis2.grid(True, which="both", alpha=0.3)
    figure2.tight_layout()
    figure2.savefig("collision_inputs.png", dpi=150)

```

```

print("Saved graphs as collision_time.png and collision_inputs.png")

```

```

def task_1c():
    print("Task 1c: Birthday collisions for truncated SHA-256")

    rows = []

    for bits in range(8, 51, 2):
        print(f"Searching for a {bits}-bit collision...", flush=True)

        input1, input2, digest, attempts, elapsed = find_collision(bits)
        rows.append((bits, input1, input2, digest, attempts, elapsed))

        print(
            f" Found after {attempts:,} inputs in {elapsed:.4f} seconds "
            f"(truncated hash: {format(digest, f'0{bits}b')})"
        )

    print()
    print(f"{Bits':>5} | {'Inputs Tried':>14} | {'Time (s)':>12}")
    print("-" * 40)

    for bits, _input1, _input2, _digest, attempts, elapsed in rows:
        print(f"{bits:>5} | {attempts:>14,} | {elapsed:>12.4f}")

    save_results(rows)

    plot_results(
        [row[0] for row in rows],
        [row[5] for row in rows],
        [row[4] for row in rows],
    )

def main():
    random.seed()
    task_1a()
    task_1b()
    task_1c()

if __name__ == "__main__":
    main()

```

crack_lib.py

```

import multiprocessing as mp
import time

```

```

import bcrypt
from nltk.corpus import words

```

```

MIN_LEN, MAX_LEN = 6, 10

```

```

def load_shadow(path):
    entries = []
    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line or ":" not in line:
                continue

```

```

        user, digest = line.split(":", 1)
        entries.append((user, digest))
    return entries

```

```

def parse_bcrypt(digest):
    _, algo, cost, tail = digest.split("$")
    return algo, cost, tail[:22], tail[22:]

```

```

def candidate_words():
    seen = set()
    out = []
    for w in words.words():
        w = w.lower()
        if MIN_LEN <= len(w) <= MAX_LEN and w not in seen:
            seen.add(w)
            out.append(w)
    return out

```

```

_WORDLIST = None
_GROUPS = None

```

```

def _init_worker(wordlist, groups):
    global _WORDLIST, _GROUPS
    _WORDLIST = wordlist
    _GROUPS = groups

```

```

def _check_chunk(task):
    group_key, lo, hi = task
    cost, salt = group_key
    salt_hash_by_tail = _GROUPS[group_key]
    salt_prefix = f"$2b${cost}${salt}".encode()
    hits = []

    for w in _WORDLIST[lo:hi]:
        digest = bcrypt.hashpw(w.encode(), salt_prefix).decode()
        tail = digest[-31:]
        if tail in salt_hash_by_tail:
            hits.append((salt_hash_by_tail[tail], w))

    return group_key, hi, hits

```

```

def _task_stream(groups, ranges):
    for lo, hi in ranges:
        any_active = False
        for key, still_unsolved in groups.items():
            if still_unsolved:
                any_active = True
                yield (key, lo, hi)
        if not any_active:
            return

```

```

def crack_all(entries, cores, chunk=200, on_found=None):
    groups = {}
    for user, digest in entries:

```

```

_, _, salt, tail = parse_bcrypt(digest)
cost = digest.split("$")[2]
groups.setdefault((cost, salt), {})[tail] = user

frozen_groups = {k: dict(v) for k, v in groups.items()}

wordlist = candidate_words()
n = len(wordlist)
ranges = [(lo, min(lo + chunk, n)) for lo in range(0, n, chunk)]

results = {
    user: {"password": None, "seconds": None, "guesses": None}
    for user, _ in entries
}

max_guesses_seen = {key: 0 for key in groups}
unsolved = len(results)
start = time.perf_counter()

with mp.Pool(
    cores,
    initializer=_init_worker,
    initargs=(wordlist, frozen_groups),
) as pool:
    for group_key, hi, hits in pool.imap_unordered(
        _check_chunk,
        _task_stream(groups, ranges),
    ):
        max_guesses_seen[group_key] = max(
            max_guesses_seen[group_key],
            hi,
        )

    for user, word in hits:
        if results[user]["password"] is None:
            results[user]["password"] = word
            results[user]["seconds"] = time.perf_counter() - start
            results[user]["guesses"] = max_guesses_seen[group_key]

        if on_found is not None:
            on_found(
                user,
                word,
                results[user]["seconds"],
                results[user]["guesses"],
            )

        tail = next(
            t for t, u in groups[group_key].items()
            if u == user
        )
        del groups[group_key][tail]
        unsolved -= 1

    if unsolved == 0:
        pool.terminate()
        break

elapsed = time.perf_counter() - start

for user, result in results.items():

```

```

        if result["password"] is None:
            result["seconds"] = elapsed
            result["guesses"] = n

    return results

```

task2_crack_real.py

```

import os
import time

from crack_lib import load_shadow, parse_bcrypt, crack_all

SHADOW_PATH = "shadow_real.txt"
RESULTS_PATH = "task2_results.txt"

def _report_progress(user, password, seconds, guesses):
    print(
        f"  cracked {user:<10} -> {password:<12} "
        f"({seconds:.1f}s, {guesses} guesses)",
        flush=True,
    )

def main():
    entries = load_shadow(SHADOW_PATH)
    cores = os.cpu_count()

    print(
        f"Loaded {len(entries)} entries from {SHADOW_PATH}; "
        f"using {cores} CPU cores.\n",
        flush=True,
    )

    start = time.perf_counter()
    results = crack_all(entries, cores, on_found=_report_progress)
    total = time.perf_counter() - start

    header = (
        f"{'User':<10} {'Algo':<5} {'Cost':<5} {'Salt':<24} "
        f"{'Password':<12} {'Time (s)':>9} {'Guesses':>9}"
    )

    lines = [header, "-" * len(header)]

    for user, digest in entries:
        algo, cost, salt, _ = parse_bcrypt(digest)
        result = results[user]
        password = result["password"] or "NOT FOUND"

        lines.append(
            f"{user:<10} {algo:<5} {cost:<5} {salt:<24} "
            f"{password:<12} {result['seconds']:>9.2f} "
            f"{result['guesses']:>9}"
        )

    lines.append("")
    lines.append(f"Total wall-clock time: {total:.2f}s")

```

```
output = "\n".join(lines)
print(output)
```

```
with open(RESULTS_PATH, "w") as f:
    f.write(output + "\n")
```

```
print(f"\nWrote results to {RESULTS_PATH}")
```

```
if __name__ == "__main__":
    main()
```